

Introduction to Machine Learning (SS 2026)

Programming Project

Author 1

Last name: Fischbach
First name: Matti Sebastian
Matrikel Nr.: 12404020

Author 2

Last name: Musch
First name: Eric
Matrikel Nr.: 12320608

I. INTRODUCTION

We address a binary classification problem: detecting fraudulent financial transactions. The dataset contains 227,845 transactions described by 31 features: Time, Amount, and 28 anonymized principal components (Feature0–Feature27) produced by a prior PCA transformation. The target variable `Class` encodes legitimate (0) or fraudulent (1) transactions.

The dataset is severely imbalanced: only 394 of 227,845 transactions are fraud (0.17%). This makes accuracy a misleading metric, since a trivial all-negative classifier achieves 99.83% accuracy while catching zero fraud. We therefore report Area Under the Precision–Recall Curve (AUPRC) as our primary metric, which directly captures the precision–recall trade-off on the minority class and is not inflated by the large number of true negatives. ROC-AUC is used as a secondary metric for leaderboard/test-dataset evaluation.

II. IMPLEMENTATION / ML PROCESS

A. Preprocessing

We applied an 80/20 stratified train-validation split to preserve the 0.17% fraud rate in both sets. The `Time` feature was dropped after empirical analysis showed it provides negligible predictive signal (AUPRC delta < 0.001). `Amount` was scaled with `RobustScaler` fitted on the training set only to prevent data leakage; this is required for Logistic Regression but omitted for Random Forest as tree models are invariant to monotone feature transformations. Class imbalance was handled via `class_weight=None` on the LR model and `class_weight=balanced` on the RF model.

B. Method 1: Logistic Regression

Logistic Regression serves as our interpretable linear baseline. We ran `GridSearchCV` (3-fold CV, scored on `average_precision`) over $C \in \{0.1, 1, 10\}$ and both L1 and L2 regularization, with `class_weight \in \{balanced, None\}`. The best configuration was L1 regularization (saga solver, implemented via `elasticnet` with `l1_ratio=1`), $C = 0.1$, `class_weight=None`.

C. Method 2: Random Forest

The Random Forest classifier uses the Decision Tree as its underlying architecture. In a decision tree we have Internal Nodes (split feature x_d with threshold θ) and Leaves which in our case are class labels (our classification). The splitting is done by reducing the **impurity** at each node. In the case of the `scikit-learn` library used, this is done via Gini impurity. The random forest uses the decision tree architecture and uses **bagging** (or bootstrap aggregation) to decorrelate the individual decision trees and reduce variance. We do this by sampling K datasets with replacement from the training dataset, then train each decision tree independently on a different bootstrapped dataset and average the predicted class probabilities across all of our K trees.

Averaging over 500 trees reduces variance substantially without overfitting, which is a key advantage over single decision trees on this high-dimensional dataset. Unlike Logistic Regression, Random Forest can capture non-linear interactions between the anonymized PCA features.

Hyperparameters were selected through five rounds of `RandomizedSearchCV` and `GridSearchCV` (3-fold CV, `average_precision` scoring), progressively narrowing the search space. The key findings were: `max_depth=None` (unlimited depth) consistently outperformed depth-limited trees because the 500-tree averaging prevents overfitting; `max_features=log2` produced more diverse trees than `sqrt` at high tree counts; and `max_samples=None` (full bootstrap sample) was confirmed optimal. Final parameters: `n_estimators=500`, `max_depth=None`, `max_features=log2`, `class_weight=balanced`.

D. Decision Threshold

For both models, the default classification threshold of 0.5 is suboptimal under heavy class imbalance. We swept the Precision–Recall curve and selected the threshold that maximizes F1-score on the validation set. This threshold is saved alongside the model artifact.

III. RESULTS

Table I summarises validation set performance for all four model variants (simple = default hyperparameters; tuned =

post-search best params).

TABLE I
VALIDATION SET PERFORMANCE

Model	AUPRC	ROC-AUC	F1	Recall
LR Simple	0.787	0.981	0.857	0.835
LR Tuned	0.807	0.983	0.827	0.848
RF Simple	0.887	0.955	0.899	0.848
RF Tuned	0.899	0.966	0.903	0.886

Figure 1 shows the confusion matrix of the tuned Logistic Regression and Random Forest models at their F1-optimal threshold ($LR_threshold \approx 0.07$; $RF_threshold \approx 0.29$). Our best model (Random Forest) catches 70 of the 79 fraud cases (recall 0.886) while raising only 6 false alarms out of 45,490 legitimate transactions (precision 0.921).

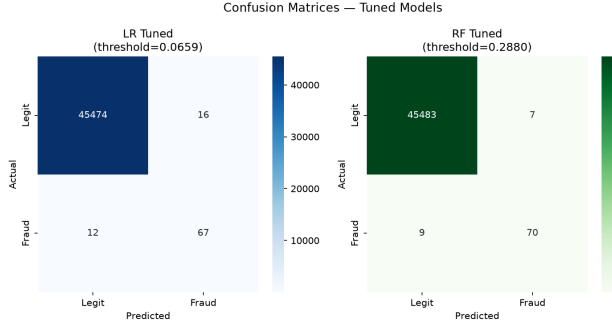


Fig. 1. Confusion matrix for LR and RF tuned.

Figure 2 shows Precision–Recall curves for all four models. The tuned RF achieves the highest AUPRC and dominates across all recall levels. Both simple models are already competitive, with tuning providing a consistent improvement of approximately +0.02 (LR) and +0.01 (RF).

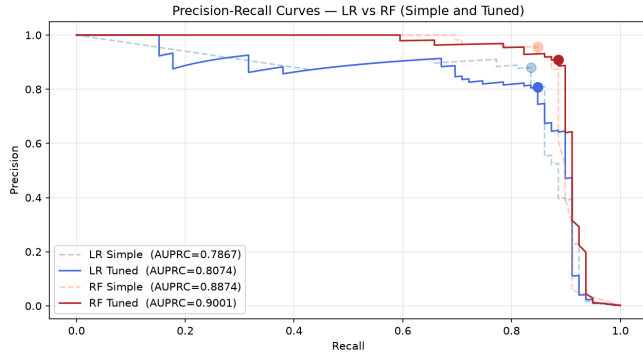


Fig. 2. Precision–Recall curves for LR and RF (simple and tuned). Dots mark the F1-optimal operating threshold per model.

Figure 3 shows the top-20 feature importances for the tuned RF model. Feature13 is clearly the most important feature, followed by three features (Feature3, Feature9, Feature4) with similar, substantially lower importance.

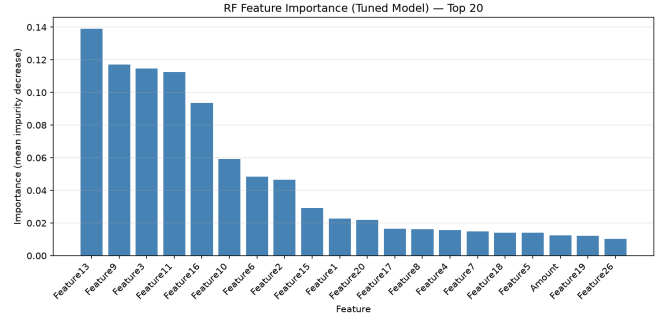


Fig. 3. RF feature importances (mean impurity decrease, tuned model).

IV. DISCUSSION

Random Forest outperforms Logistic Regression substantially in AUPRC (+0.09), which we attribute to non-linear interactions among the PCA features that a linear model cannot capture. The high ROC-AUC (0.966 val / 0.972 test) confirms the model generalises well to unseen data.

What was tried and discarded: (1) Including Time as a feature (AUPRC delta was <0.001), so it was dropped to simplify the pipeline. (2) A Neural Network was initially considered as the second method but replaced by Random Forest, which requires no architecture design and achieves better out-of-the-box performance on tabular data. (3) During tuning, accidentally omitting `class_weight=balanced` produced the fewest false positives (3) but lower recall than the balanced variant (68/79 fraud caught). This further confirms that balanced weighting is critical for this highly imbalanced dataset. (4) Tuned LR achieves higher AUPRC (0.807 vs. 0.787) but lower F1 (0.827 vs. 0.857) than simple LR. Stronger regularization ($C = 0.1$, L1) improves probability calibration across all thresholds (higher AUPRC) but constrains the peak F1 attainable at the optimal operating point.

Potential improvements: Gradient boosting methods (XGBoost, LightGBM) may further improve AUPRC. Probability calibration could refine threshold selection.

V. CONCLUSION

We implemented two fraud detection classifiers, Logistic Regression and Random Forest, on a severely imbalanced financial transactions dataset (0.17% fraud). The tuned Random Forest achieves an AUPRC of 0.899 on the local validation set and a ROC-AUC of **0.9716** on the hidden test set, demonstrating strong generalisation. Our main takeaways are: (1) AUPRC is the appropriate metric for heavily imbalanced binary classification; (2) For RF, `class_weight=balanced` is essential for recall on the minority class; LR’s grid search found `class_weight=None` optimal for AUPRC. (3) Random Forest with unlimited tree depth and 500 estimators significantly outperforms a tuned linear baseline, capturing non-linear structure in the anonymised PCA features without requiring feature scaling or resampling.